

Project Progress Report

Data Warehouse

Bachelor of Science in Data Science, Faculty of Mathematics and Natural Sciences
Surabaya State University

Class	2024-INT
Leader	Daffa Ahmad Pangreksa (24031554159)
Member 1	Halilatunnisa (24031554130)
Member 2	Naura Kanaya Putri M. (24031554138)
Github URL	dapp-eng/talent-trackk

General Project Information

Title	TalentTrack: A Multi-Platform Job Posting Data Warehouse with NLP-Based Skill and Position Extraction for Labor Demand Trend Analysis
Topic	Social
Revision?	Yes/No

Description of the Research Formulation	The global labor market has undergone rapid structural transformation driven by digitalization, automation, and the post-pandemic economic recovery. Employers increasingly demand specific and often highly technical skill sets, yet job seekers and academic institutions lack timely, data-driven intelligence on which skills and positions are actually in demand at any given moment. Traditional labor market reports published by government agencies or research institutions are typically released quarterly or annually, making them too slow to capture fast-moving hiring trends that can shift within weeks. The core problem is an information asymmetry between the supply side of labor (job seekers, students, career counselors, educators) and the demand side (employers, recruiters). A fresh graduate in data science, for instance, has no reliable way of knowing whether the market currently prioritizes Python and cloud skills over classical statistics, or whether mid-level machine learning engineer roles are concentrating in specific industries or geographies at this moment in time. This problem persists because job posting data, while publicly available on platforms such as LinkedIn, Indeed, and Glassdoor, is unstructured, scattered across multiple sources with inconsistent formats, and not aggregated into any analytically consumable form for trend analysis. Each platform presents postings in isolation and optimizes for job matching rather than market intelligence. Furthermore, the textual nature of job descriptions means that skill and position information is embedded in free-form language, requiring natural language processing pipelines to extract structured signals from them. The challenge of solving this problem therefore lies in three dimensions: first, the technical difficulty of integrating and harmonizing large-scale historical datasets with periodically acquired multi-platform job posting data; second, the computational complexity of applying NLP models such as JobBERT or Sentence-BERT for semantic feature extraction and Named Entity Recognition (NER) for extracting structured skill and position entities from unstructured text; and third, the analytical design challenge of constructing a hybrid data warehouse and semantic retrieval pipeline capable of generating multidimensional trend insights across time, geography, industry, platform, and skill relationships simultaneously.
---	--

Target Insights to be Obtained	The primary insights targeted through the Data Staging and Data Presentation pipeline using CUBE queries are as follows: - The first target insight is identifying which technical and soft skills are experiencing the highest growth in demand on a weekly and monthly basis, disaggregated by job categories such as Data Science, Software Engineering, Finance Technology, and Cloud Infrastructure. This reveals which skills are currently surging versus declining in the market. - The second target insight is understanding the geographic concentration of demand for specific job positions and experience levels. By aggregating across the location and position dimensions with time granularity, the system will surface whether senior data roles are concentrating in certain countries or cities in a given month compared to the previous period. - The third target insight is platform behavior comparison, examining whether LinkedIn, Indeed, and Glassdoor exhibit systematic differences in the types of roles, seniority levels, and industries they represent, which informs users on where to search depending on their profile. - The fourth target insight is the relationship between skill combinations and salary ranges, using the salary information available primarily through Indeed and Glassdoor postings, to understand whether certain skill clusters command measurably higher compensation across job levels. - The fifth target insight is temporal hiring velocity by industry and company size, identifying which industries and company size segments are accelerating or decelerating their hiring activity on a quarterly basis, serving as a leading indicator of sectoral economic activity.
Research References	https://doi.org/10.1016/j.eswa.2024.124101 https://doi.org/10.18653/v1/2022.naacl-main.366 https://doi.org/10.1186/s40537-022-00576-5 https://doi.org/10.1186/s40537-022-00649-5

Detailed Dataset Information

Data Source	Data acquisition combines two complementary sources. The first source is a sizable historical dataset that was acquired from Kaggle and includes over 124,000 LinkedIn job postings that were gathered between 2023 and 2024. Job titles, descriptions, salaries, employment types, locations, application URLs, experience levels, company details, employee numbers, benefits, and necessary abilities are all included in this collection. The dataset was originally collected using the LinkedIn Job Scraper and serves as the primary foundation for historical labor market analysis. The second source is periodic incremental extraction using JobSpy, an open-source Python module that uses a uniform API interface to dynamically obtain publicly accessible job posts from Indeed, LinkedIn, Glassdoor, ZipRecruiter, and Google Jobs. With fairly equal sampling across supported platforms and search phrases, the scraping pipeline targets about 100 new job listings per extraction cycle and is set to run every three days. The periodic extraction process is intended to support continuous dataset updates and temporal trend monitoring during system operation. Raw outputs from both the historical dataset and the periodic scraping pipeline are stored as timestamped CSV or Parquet files before entering the preprocessing and transformation stages of the analytical pipeline.
General Description of The Dataset / Data Dump	Each record in the dataset corresponds to a single job posting obtained either from the historical Kaggle dataset or from periodic incremental extraction using JobSpy. The available fields vary across platforms but generally include job title, company name, company location, job description text, posting date, employment type (full-time, part-time, contract), remote-work availability, platform source, and, in a subset of postings, salary ranges expressed as minimum and maximum values when available. From the unstructured job description text, the transformation pipeline derives

	additional structured representations. First, semantic embedding vectors are generated using pre-trained JobBERT and Sentence-BERT models to capture contextual similarity between job postings. Second, structured skill entities and normalized job position information are extracted using a rule-based entity extraction pipeline with manually curated skill mappings and pattern matching techniques during the current prototype stage. Future development is planned to integrate transformer-based knowledge extraction and NER models for improved contextual entity recognition. The historical dataset contains more than 124,000 job postings collected throughout 2023 and 2024, while the periodic extraction pipeline continuously adds new postings from multiple online recruitment platforms during system operation. After preprocessing, deduplication, and NLP-based transformation, the final dataset is intended to support OLAP analysis, semantic similarity retrieval, labor market trend analysis, and forecasting of skill demand patterns.
Specific Aspects of The Dataset	The dataset exhibits notable sparsity in its salary-related fields. Across all platforms, salary information is available only in a subset of postings because many employers do not disclose compensation details publicly. As a result, salary-related measures in the fact table contain a substantial proportion of null values, requiring salary-based OLAP analysis to focus primarily on postings with available salary information. The dataset is also moderately imbalanced across job categories. Technology-related roles such as Software Engineering and Data Science dominate posting volumes relative to Finance Technology or Business Intelligence positions. In addition, postings originating from North American and European markets significantly outnumber postings from Southeast Asian regions due to the composition of the historical dataset and the dominance of English-language recruitment platforms. Salary values present the primary outlier risk. Some postings contain unusually high salary ranges caused by data entry inconsistencies, inclusion of equity compensation, or differing salary period formats. The transformation pipeline applies interquartile range (IQR)-based filtering segmented by job level and country to detect and exclude salary outliers before loading. Duplicate postings also represent a major data quality issue. The same vacancy may appear across multiple recruitment platforms or exist simultaneously in both the historical dataset and periodic extraction results. Cross-source deduplication is performed using company name, job title similarity, posting date proximity, and cosine similarity between semantic embedding vectors to identify highly similar postings before retention of the first valid occurrence.

Data Staging Progress

Extraction Stage	Extraction Stage The extraction module in TalentTrack consists of two independent but complementary pipelines, both managed by an Apache Airflow DAG scheduled to run every three days. - The first pipeline handles historical batch extraction from the Kaggle 2024 dataset, which is stored locally as a CSV file at data/raw/kaggle_jobs_2024.csv. The extractor reads the file in chunks of 50 rows at a time using pandas to avoid loading the entire dataset into memory at once. Each chunk goes through column normalization, where field name variants such as job_title, position, or title are mapped to a canonical 10-field schema covering title, company, location, description, posting date, salary range, remote status, platform, and employment type. Every row receives an MD5 hash generated from the combination of company name, job title, and posting date, which serves as the primary duplicate detection key. A batch limit of 150 rows per run is enforced to control processing load in each Airflow cycle. The final output is written as a Parquet file alongside a
------------------	--

	metadata JSON file recording the row count, extraction timestamp, and batch limit. - The second pipeline performs periodic scraping using JobSpy, an open-source Python library that queries LinkedIn, Indeed, Glassdoor, ZipRecruiter, and Google Jobs through a unified API. Each scheduled run iterates over up to 5 search terms from a configured vocabulary of over 50 job titles, across up to 4 geographic locations drawn from a list of over 60 global markets. A hard cap of 100 unique postings per run is applied. A random delay of 1.5 to 3.0 seconds is introduced between each scraping request to reduce the risk of rate limiting. Column alignment and hash generation follow the same logic as the Kaggle pipeline. If JobSpy is unavailable in the execution environment, the pipeline writes an empty Parquet file and logs the skip, allowing the Airflow DAG to continue without failure. The resulting data is saved as a timestamped Parquet file in the raw periodic directory and simultaneously copied to the processed directory for downstream use. Both pipelines are wrapped as Airflow tasks using ExternalPythonOperator, executing inside a dedicated virtual environment to isolate dependencies. The DAG initializes the PostgreSQL schema via DDL on the first run and seeds the dim_time dimension table with daily entries from January 2024 through December 2027 before any extraction task begins.
Transformation Stage	The transformation pipeline applies three sequential processes to each batch of records before they are loaded into the data warehouse. - The first process is general preprocessing, implemented in transform/preprocess.py. Job title, description, and company name fields are cleaned by stripping HTML tags, normalizing Unicode characters to their ASCII equivalents, removing URLs, and collapsing whitespace before converting all text to lowercase. Posting dates are parsed with a custom function that recognizes both standard date formats and relative expressions such as "3 days ago", "last week", "just posted", and "30+ days ago". Records with parsed dates outside the range of January 2022 to the current date are discarded. Location strings are parsed into four structured components: city, province or state, country, and global region. The global region is assigned through keyword matching against a lookup map covering over 50 countries across North America, Europe, Southeast Asia, South Asia, East Asia, Oceania, South America, Africa, and the Middle East. Job level is inferred from the job title by keyword matching across four tiers: Junior, Mid, Senior, and Manager. Job category is similarly inferred using keyword matching against 10 predefined categories including Data, Machine Learning, Cloud, DevOps, FinTech, Engineering, Management, Design, Security, and Database. Salary values are numerically parsed from varied formats including currency symbols and shorthand notation such as "80k", and are then validated and filtered against interquartile range bounds segmented by job level to exclude outliers. Platform names are standardized to a canonical set covering LinkedIn, Indeed, Glassdoor, ZipRecruiter, and Google Jobs. Records with a title shorter than 2 characters or a description shorter than 20 characters are removed before the batch proceeds. - The second process is semantic embedding generation, implemented in transform/embed.py. Two pre-trained models are applied to each cleaned job description. The first model is JobBERT (jjzha/jobbert-base-cased), a domain-specific BERT variant trained on job-related text, which produces 768-dimensional embedding vectors. Because job descriptions often exceed the 512-token input limit, a sliding window chunking strategy is applied with a stride of 446 tokens and an overlap of 64 tokens, and the final embedding is computed as a mean pool across all chunk embeddings. GPU acceleration is used automatically when available. The second model is Sentence-BERT (all-MiniLM-L6-v2), which

	produces 384-dimensional sentence-level semantic embeddings used for lightweight similarity comparison. Both embedding arrays are saved as NumPy compressed files indexed by source hash, where each hash maps to its corresponding embedding row. - The third process is skill extraction using a rule-based Named Entity Recognition approach, implemented in <code>transform/ner_skills.py</code>. A curated vocabulary of approximately 90 skills is defined, covering programming languages such as Python, SQL, Java, and Go; big data frameworks such as Spark, Kafka, and Airflow; cloud platforms including AWS, GCP, and Azure; machine learning frameworks including TensorFlow, PyTorch, scikit-learn, and Hugging Face; DevOps tools such as Kubernetes, Docker, and Terraform; business intelligence tools including Power BI, Tableau, and Looker; and soft skills such as Communication, Leadership, and Problem Solving. Each skill is matched against the cleaned description text using a pre-compiled regular expression with word boundary constraints to avoid partial matches. Every matched skill is recorded with its associated skill type and domain, and a fixed extraction confidence score of 0.90. The output is a long-format DataFrame with one row per skill-posting pair, ready to be loaded into the bridge table. Deduplication across batches is handled in <code>transform/dedup.py</code> using a two-stage approach. The first stage performs hash-based exact matching to remove records already present in the database. The second stage performs embedding-based similarity deduplication within the new batch: cosine similarity is computed between all pairs of SBERT vectors, and a record is marked as a duplicate if its cosine similarity to an earlier record exceeds 0.92 and the two records share either the same company name or a title word-overlap score above 0.80. Only the first occurrence is retained.
Load Stage	The load module writes the fully transformed and deduplicated records into PostgreSQL using a structured and conflict-safe upsert strategy implemented in <code>load/load_dimensions.py</code> and <code>load/load_facts.py</code>. All six dimension tables are populated before the fact table to ensure referential integrity. Each dimension uses an <code>INSERT INTO ... ON CONFLICT DO NOTHING</code> statement, which allows incremental loading without duplication. After each dimension is upserted, the full mapping from natural keys to surrogate IDs is retrieved in a single query and held in memory as a dictionary for fast fact row construction. The <code>dim_time</code> table is seeded in advance by the Airflow DAG initialization task, covering all dates from 2024 through 2027. The central fact table, <code>fact_job_posting</code>, is partitioned by <code>time_id</code> into four child partitions covering 2024, 2025, 2026, and future dates respectively. Partitioning improves query pruning performance for time-bounded OLAP queries. Each fact row stores foreign keys to all five dimension tables along with measures for posting count, job age in days, salary minimum, salary maximum, and boolean flags for salary availability and remote status. Fact rows are bulk-inserted using <code>psycopg2.extras.execute_values</code> with a page size of 500 rows per batch, and conflicts on the <code>(source_hash, time_id)</code> unique key are silently ignored to prevent duplicates from incremental runs. The bridge table <code>bridge_job_skill</code> is loaded after the fact table using the job ID mappings retrieved from the database. It links each job posting to each of its extracted skills with a recorded extraction confidence value. Bridge inserts use a page size of 1,000 rows per batch. Semantic embedding vectors are stored in a dedicated <code>job_embeddings</code> table using the <code>pgvector</code> PostgreSQL extension. Both the 768-dimensional JobBERT vectors and the 384-dimensional SBERT vectors are stored per job and indexed with IVFFlat

	approximate nearest neighbor indexes using 100 list partitions, which enables sub-second cosine similarity retrieval for semantic job matching at scale. Three materialized views are defined in the DDL to pre-aggregate the most frequently queried combinations. The <code>mv_weekly_skill_demand</code> view joins the fact table with the bridge, skill, time, position, and location dimensions, grouping results by week and skill for fast weekly trend access. The <code>mv_platform_monthly</code> view aggregates posting volume, remote rate, and salary availability rate by platform and month. The <code>mv_company_hiring</code> view aggregates quarterly hiring counts and average salary by company, industry sector, and company size. All three views support concurrent refresh through unique indexes, preventing read blocking during updates. Technology used: PostgreSQL serves as the primary relational storage engine. The <code>pgvector</code> extension provides native high-dimensional vector storage and approximate nearest neighbor search. The <code>pg_trgm</code> extension enables trigram-based GIN indexing on company names for fuzzy text search. The <code>btree_gin</code> extension supports composite GIN indexes. Application-level access uses <code>psycopg2</code> for direct cursor-based operations and <code>SQLAlchemy</code> for engine management in analytical query execution. The full pipeline is orchestrated through Apache Airflow with <code>ExternalPythonOperator</code> tasks isolated in a dedicated virtual environment. Database size and performance benchmarking: The schema and all DDL objects including tables, indexes, partitions, and materialized views are fully deployed. End-to-end performance benchmarking results and the cumulative database size figure are pending the completion of the ongoing full-scale historical data ingestion run, which is currently being executed against the Kaggle 2024 dataset.
--	--

Progress Data Presentation

Subject focus	This project's OLAP subject focus is job market demand intelligence, specifically the multidimensional and temporal dynamics of skill and job position demand obtained from online job advertisements gathered across many platforms. Finding the talents, technologies, and job positions that employers most want, as well as how these demands change over time, across geographies, sectors, and job categories, is the main analytical goal. Multidimensional analysis is intended to be supported by the system for: - Trends in skill demand - The distribution of employment positions - Firm hiring practices - The geographic distribution of jobs - The connections between roles and necessary skills The analytical procedure makes use of preprocessed structured job posting data, semantic feature extraction using JobBERT/SBERT embeddings, rule-based entity extraction for identifying skill entities and job positions during the current prototype stage, alongside vector-based semantic retrieval for similarity analysis between job postings. Future development is planned to incorporate transformer-based NER and knowledge extraction models for improved contextual entity recognition.
Star schema	The star schema in this project is organized around <code>fact_job_posting</code> as the central fact table that serves as the core of OLAP analysis. This table is connected to several dimension tables through integer foreign key relationships to support multidimensional analysis of labor demand trends. The schema is designed to enable analysis based on time, geographic distribution, job position, recruitment platform, company information, and required skills.

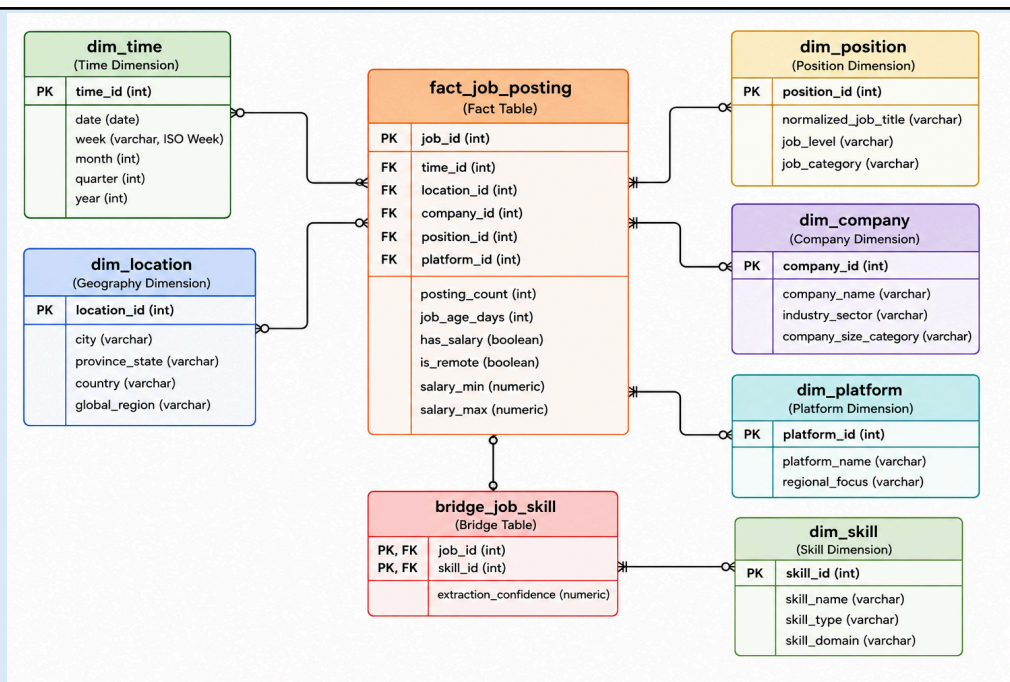


Figure 1. Star schema design for OLAP-based labor demand analysis

1. **The fact_job_posting table** functions as the primary storage for measures and analytical indicators used in the OLAP process. A distinct job posting that has been preprocessed and deduplicated is represented by each row in the table. This table's primary metrics, which facilitate aggregation and multidimensional analysis, are posting_count, job_age_days, has_salary, is_remote, salary_min, and salary_max.
2. **The dim_time table** serves as the time dimension that supports temporal analysis of job posting data. The hierarchy in this table consists of Date → Week → Month → Quarter → Year, allowing analysis to be performed from daily to yearly granularity levels. In addition, the table stores a computed ISO-format week label that is used as the primary join key in the weekly skill demand materialized view.
3. **The dim_location table** functions as the geographic dimension and stores location-related information for each job posting. Geographic roll-up analysis from city-level detail to regional and global comparisons is made possible by this table's structure, which is City → Province/State → Country → Global Region. North America, Europe, Southeast Asia, South Asia, East Asia, Oceania, South America, Africa, Middle East, Remote/Global, and Other are among the designated global regions.
4. **The dim_position table** represents the job position dimension and stores normalized job title information. The two primary analytical attributes in this table are job_level and job_category. The job level hierarchy consists of Junior, Mid, Senior, and Manager levels, allowing labor demand analysis to be conducted based on seniority and functional job categories.
5. **The dim_company table** stores metadata related to companies that publish job postings. Important attributes in this table include company_name, industry_sector, and company_size_category, which are mainly used in company hiring velocity analysis. The table also uses a GIN trigram index on company names to support approximate text lookup during the data loading phase.
6. **The dim_platform table** stores information regarding the recruitment platform from which each posting originates, such as LinkedIn, Indeed, Glassdoor, and Google Jobs. The main attributes in this table are platform name and

	regional_focus. This dimension is used to compare recruitment activity, salary disclosure rates, and remote work availability across different job platforms. 7. The dim_skill table and bridge_job_skill table are used together to model the many-to-many relationship between job postings and required skills. One job advertisement may list more than one skill, and multiple posts may list the same skill. The bridge_job_skill table stores the link between job_id and skill_id together with the extraction_confidence value, whereas the dim_skill table provides descriptive characteristics like skill_name, skill_type, and skill_domain that are used as hierarchy levels in the SkillDemand OLAP cube. In addition to the fact and dimension tables, the system also uses three materialized views such as mv_weekly_skill_demand, mv_platform_monthly, and mv_company_hiring. These materialized views serve as the primary data sources for Atoti cube construction and help reduce query execution time by avoiding repeated joins across large tables. The use of pre-aggregated views significantly improves the efficiency and responsiveness of OLAP query processing.
OLAP Result	The OLAP query layer is implemented in analysis/olap_queries.py and defines six analytical query functions operating on the star schema and materialized views. The primary query applies a full CUBE operator across 12 dimension columns including time, skill, job category, job level, geographic region, country, and platform dimensions. This enables multidimensional slice-and-dice analysis without manually rewriting aggregation queries. Additional queries retrieve top-demanded skills per category, weekly skill demand trends, platform comparisons, salary statistics by skill and job level, and forecasted skill demand for the next 8 weeks with prediction intervals. The forecasting module currently uses baseline statistical models from the statsmodels library for prototype validation and OLAP integration testing. The current implementation focuses on validating temporal aggregation and forecast visualization workflows rather than maximizing predictive accuracy. Future development is planned to introduce an adaptive forecasting strategy based on the amount of historical weekly data available: - Less than 8 weeks: forecasting skipped due to insufficient data - 8–15 weeks: ARIMA forecasting - 16–51 weeks: Holt-Winters forecasting - 52 weeks or more: Prophet-based forecasting The Atoti session defined in atoti_app/datamart.ipynb constructs three OLAP cubes: JobPostings, SkillDemand, and SkillForecast, which are exposed through an interactive web interface on localhost port 9090. The cubes support multidimensional analysis across time, location, position, platform, and skill hierarchies together with measures such as posting count, remote-work rate, salary coverage, and salary statistics. The SkillForecast cube currently presents forecast outputs generated from the prototype forecasting module, with future plans to integrate adaptive model selection for improved seasonal trend modeling and forecasting robustness.
OLAP Visualization	Visualization outputs are being prepared within the Atoti web interface, which renders pivot tables and charts directly from the in-memory OLAP session. The planned visualizations include a weekly trend line chart for top-10 skills by posting volume, a heatmap of skill demand by global region and job category, a bar chart comparing platform posting volumes and salary disclosure rates, a scatter plot of salary distribution by skill and job level, and a line chart showing forecasted skill demand trends with confidence intervals. The current forecast visualizations are generated using baseline statistical forecasting outputs from the prototype forecasting module implemented with the statsmodels library. These visualizations are primarily intended to validate temporal aggregation

	and OLAP integration workflows during the current development stage. Future development is planned to incorporate adaptive forecasting model selection strategies, including ARIMA, Holt-Winters, and Prophet models depending on historical data availability and seasonal coverage. All visualization outputs will be exported from the Atoti session after completion of the data loading and pipeline stabilization process and included in the final report as supplementary figures.
--	--